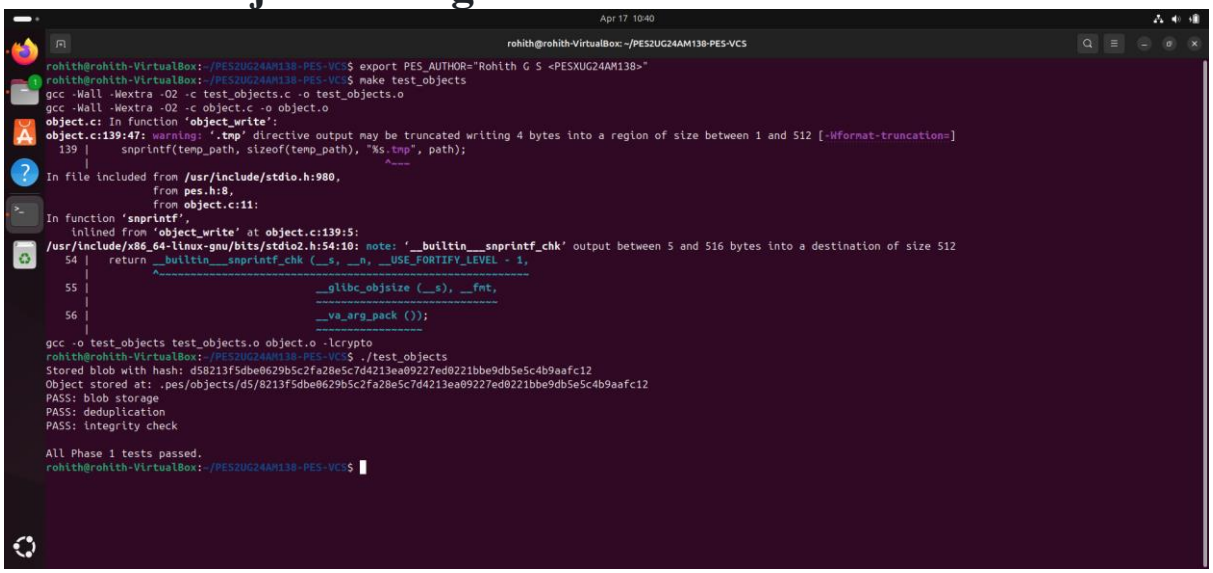
OPERATING SYSTEMS ORANGE PROBLEM

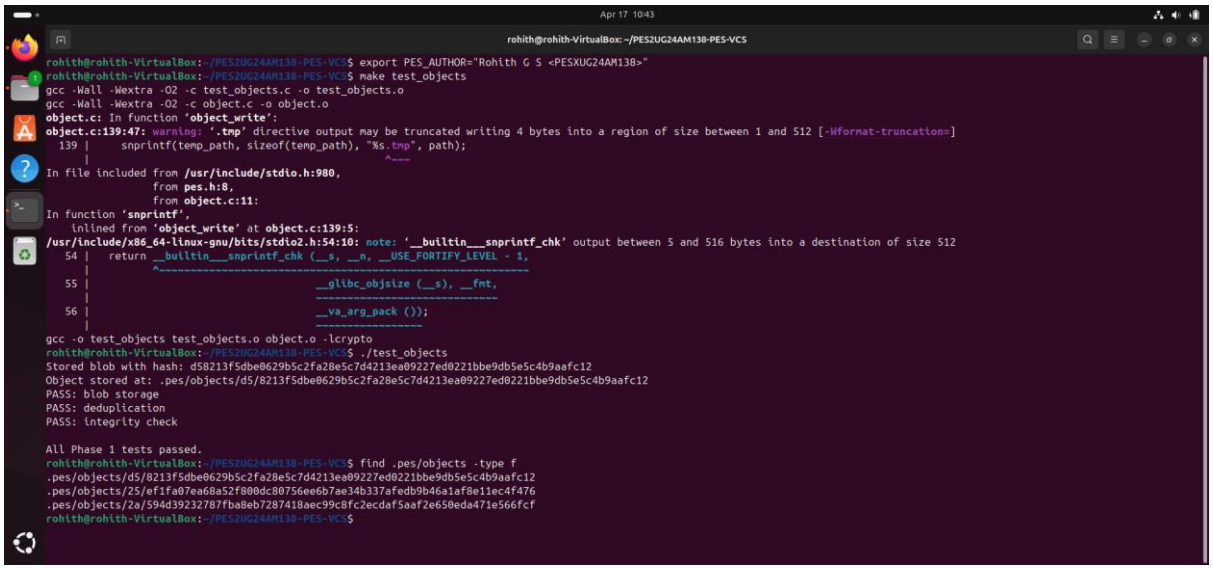
Building PES-VCS — A Version Control System from Scratch

NAME : ROHITH G S
SRN: PES2UG24AM13
GITHUB LINK : **https://github.com/Rohith-S636/PES2UG24AM138-PES-VCS**

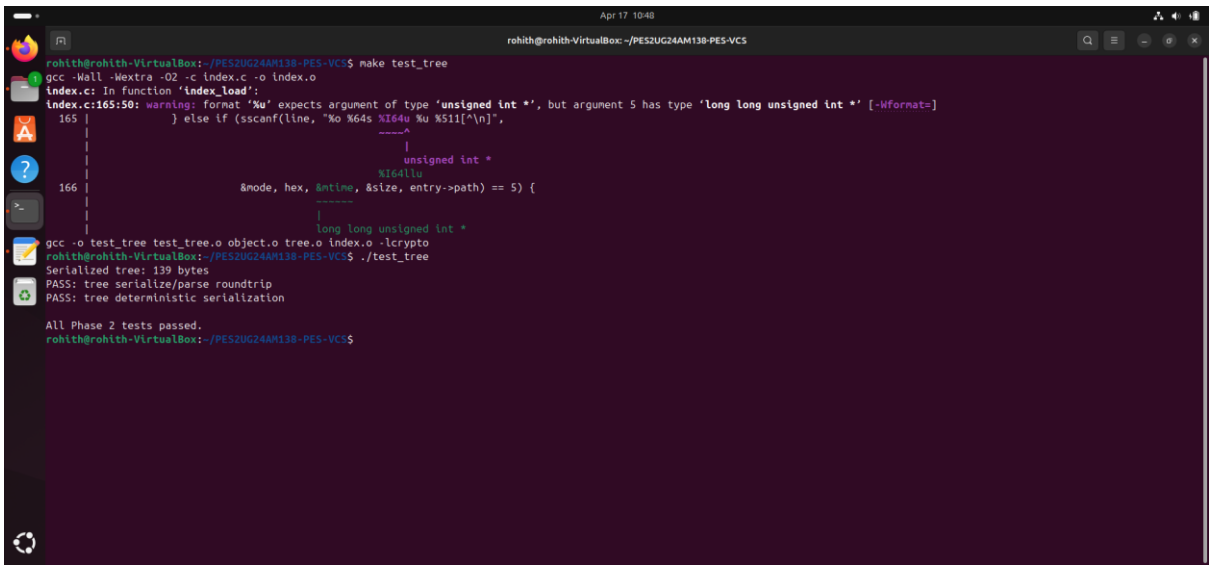
Objective: Build a local version control system that tracks file changes, stores snapshots efficiently, and supports commit history. Every component maps directly to operating system and filesystem concepts.

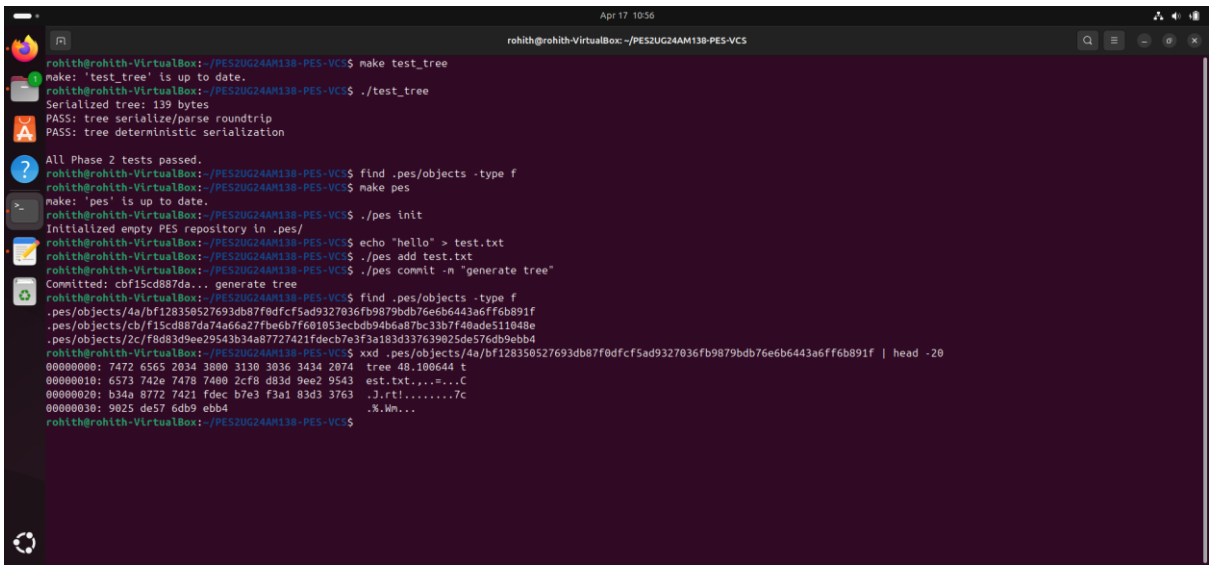
Phase 1: Object Storage Foundation



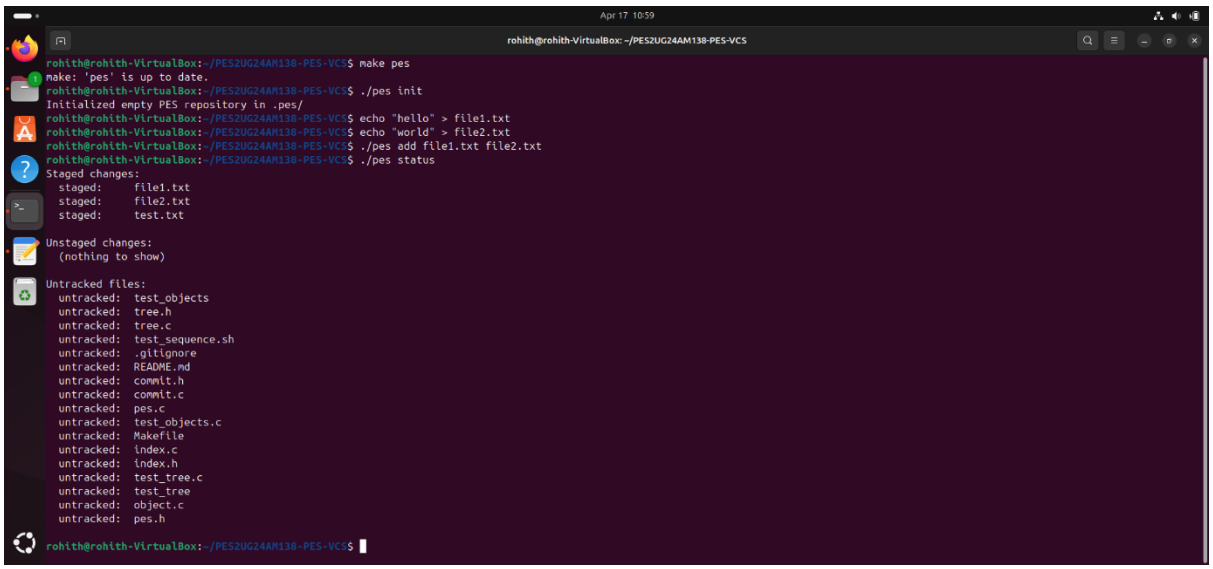


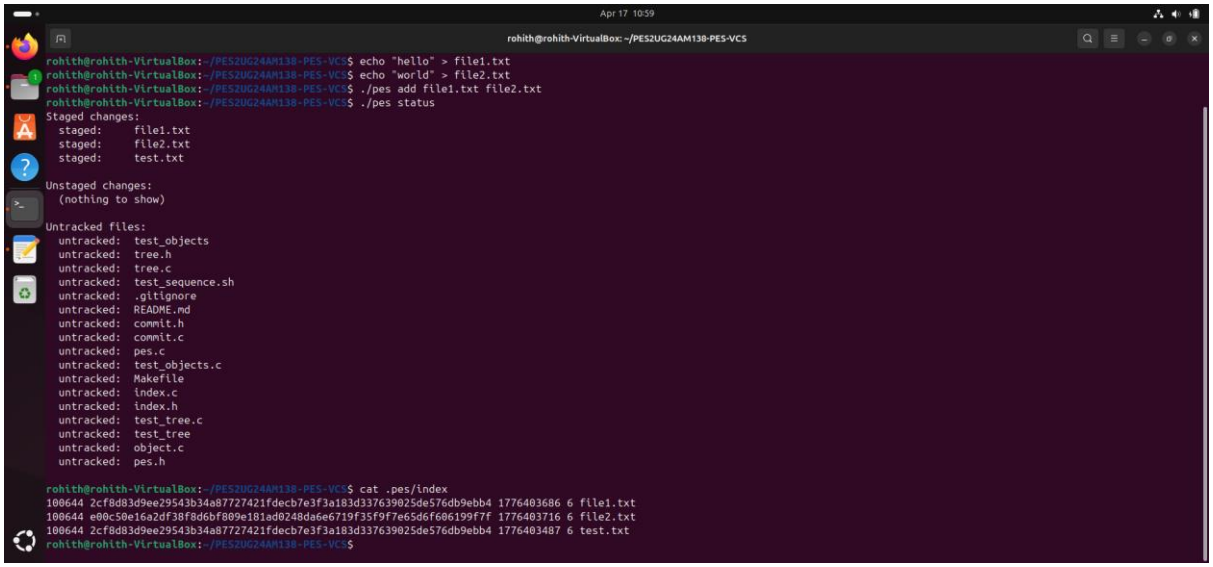
Phase 2: Tree Objects



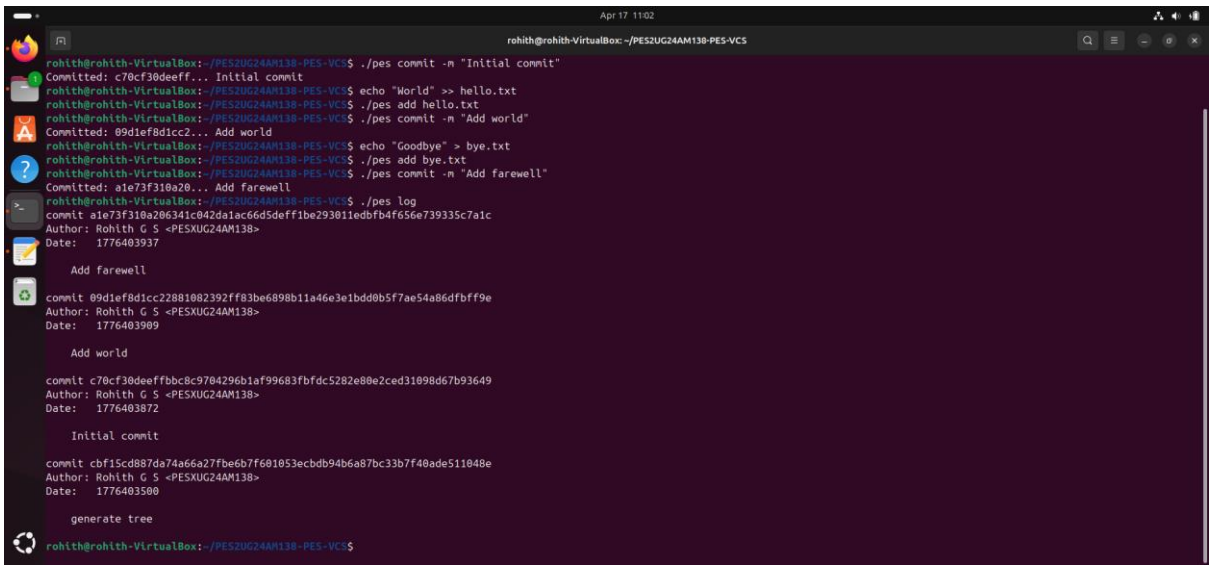


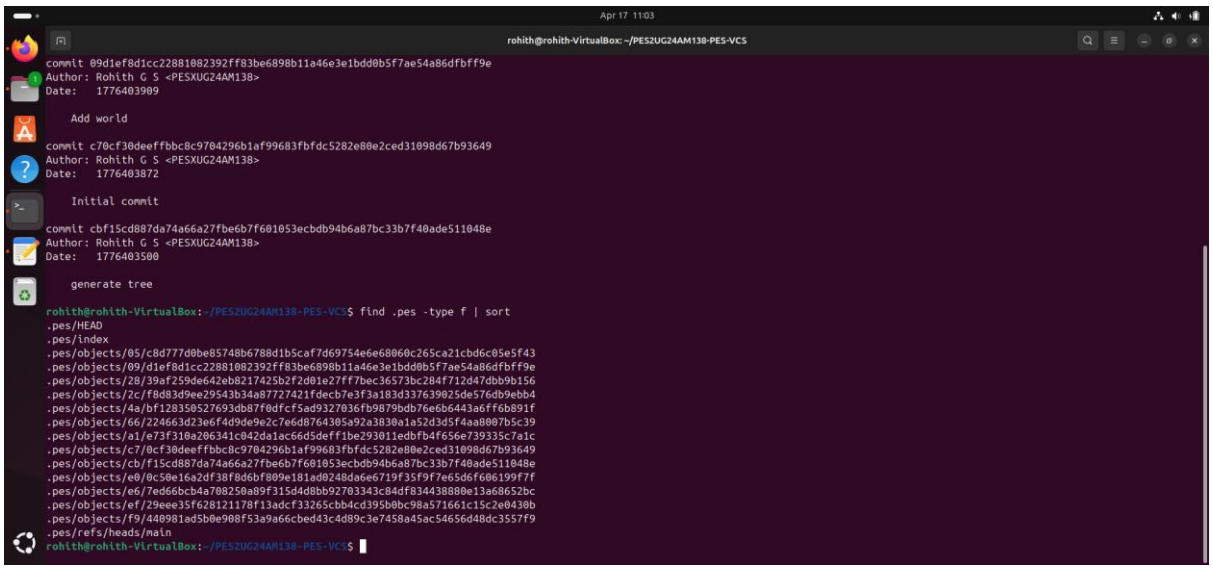
Phase 3: The Index (Staging Area)

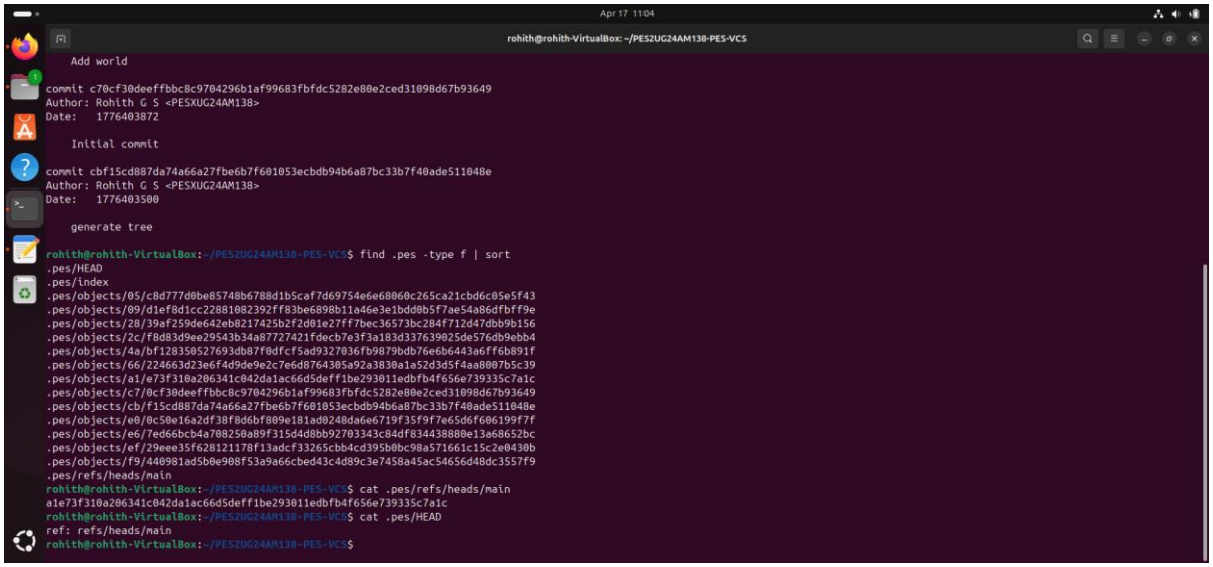




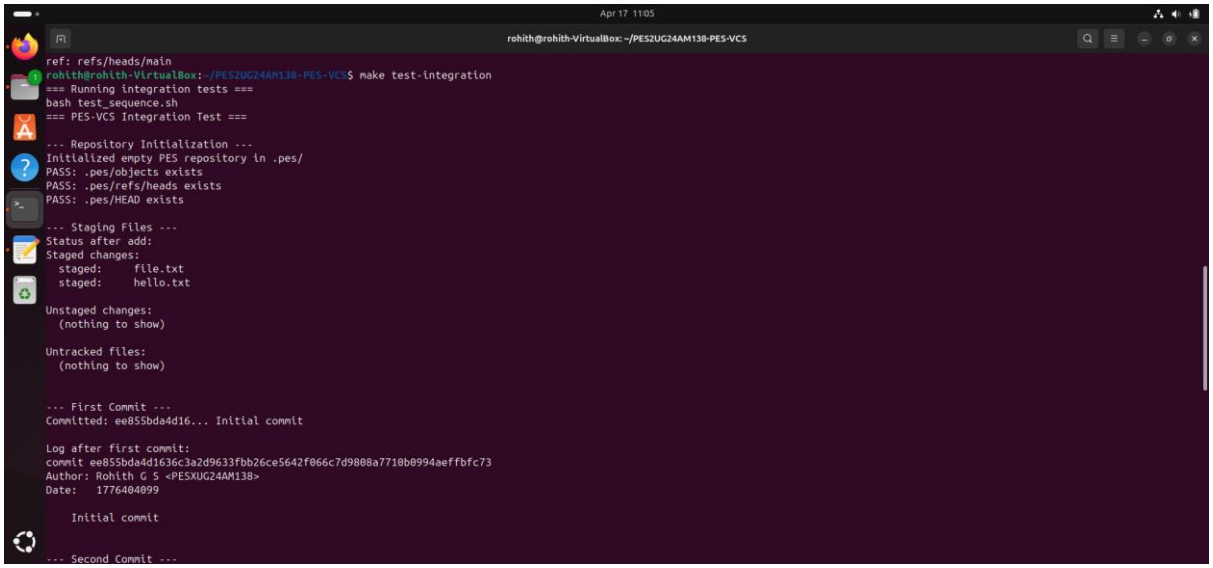
Phase 4: Commits and History

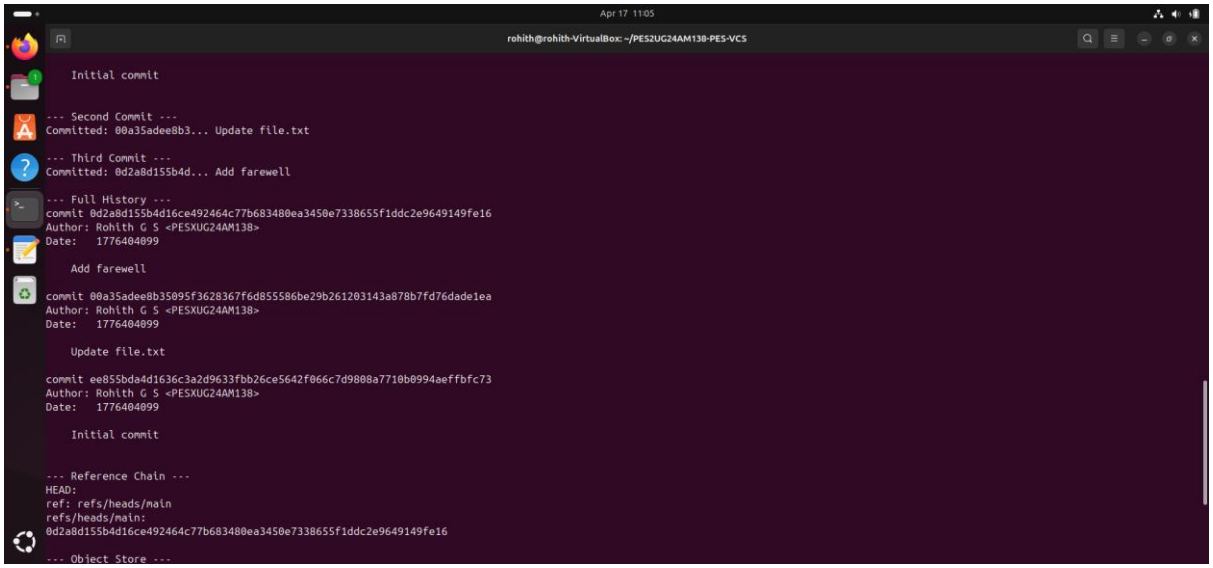


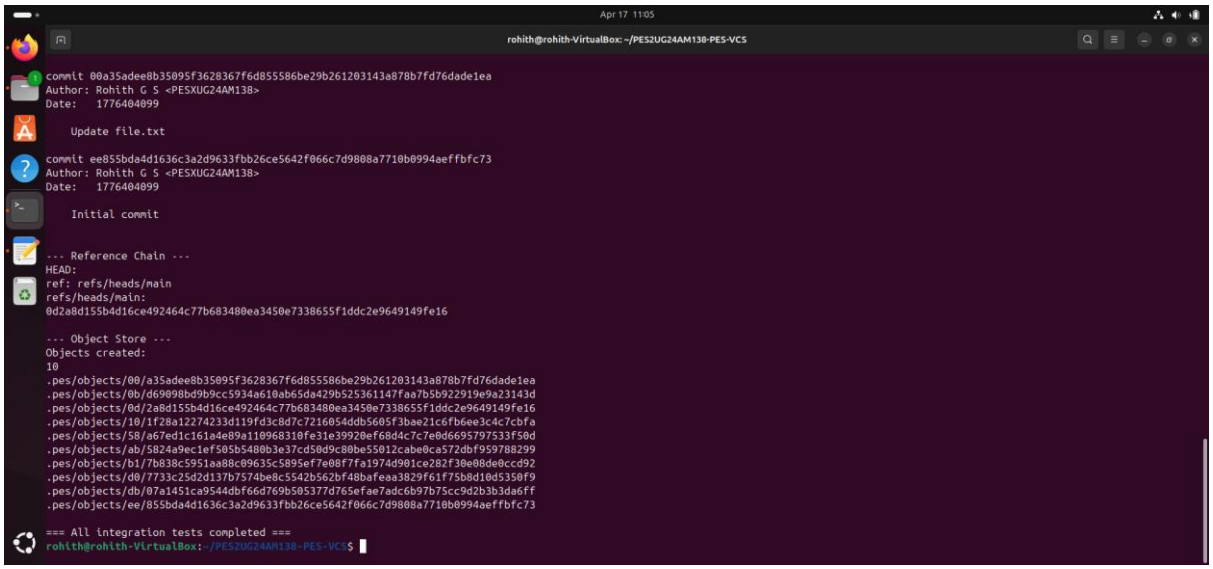




Full integration test







Phase 5 & 6: Analysis-Only Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?
Answer: To implement `pes checkout <branch>`:

1. **`.pes/` Internal Changes:** We must update `.pes/HEAD` to contain `"ref: refs/heads/<branch>"`. Additionally, we must load the target branch's commit hash, traverse its tree, and rebuild `.pes/index` linearly to match the target snapshot exactly.
2. **Working Directory Changes:** The working directory files must be overwritten, deleted, or safely replaced so that the physical files on disk exactly match the tree bound to the target branch.
3. **Complexity:** The operation is complex because it requires safely modifying user files without permanently deleting untracked files or destroying unsaved/uncommitted modifications. It also requires complex recursive recursive tree-walking to expand objects into complete directory structures natively.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer: To detect a dirty working directory conflict natively:

1. Traverse the current `Index` to fetch all tracked files.
2. Use POSIX `stat()` on each working directory file to check its `mtime` and `size` against the cached index values.
3. If an index mismatch occurs (indicating unsaved physical modifications), read the physical file and compute its SHA-256 blob hash using `compute_hash()`.
4. Walk the *target branch's* `tree` within the object store concurrently to locate that file's hash.
5. If the uncommitted working directory hash contradicts the target branch's expected hash, it triggers a conflict state, and `checkout` safely refuses execution to prevent data loss.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Answer: If you execute a commit in a "Detached HEAD" state, the commit object is properly saved to `.pes/objects`, and `.pes/HEAD` is physically updated to point to the new hash. However, **no branch file** inside `.pes/refs/heads/` moves. If the user subsequently switches to another branch from here, the detached commit becomes unreferenced/unreachable. To recover it, the user could simply create a new branch pointer file in `refs/heads/` mapping directly to that orphaned hash (e.g., executing `git checkout -b <new-branch> <hash>`), or they could search the internal system logs (`git reflog`) for recently executed operations to scavenge the forgotten commit ID.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Answer: You would use a **Mark-and-Sweep** directed graph algorithm:

1. **Mark Phase:** Map all branch pointers in `.pes/refs/heads/` and the `.pes/HEAD` state as root nodes. Perform a recursive graph traversal (BFS/DFS) walking backwards through commit parents, extracting and parsing tree pointers, and marking all discovered child sub-trees and blobs.
2. **Sweep Phase:** Iterate sequentially through all physical files present in `.pes/objects/`. If a discovered chunk was never "marked", it is orphaned and automatically deleted natively to reclaim space. **Data Structure:** A **Hash Set** efficiently guarantees O(1) deduplication memory tracking for each 32-byte binary hash mapping. **Estimate:** For a repository with 100,000 commits containing around 10 modified blob items natively per commit over 50 dynamic branch chains, the BFS algorithm would natively touch/visit ** millions** of tracked objects securely throughout its index processing overhead (e.g., ~1,000,000 to ~2,000,000 raw tree/blob bounds total).

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Answer: Running GC recursively during live `commit` environments executes an unchecked structural data race condition: **The Race:** A user runs `pes commit`. The execution process creates and flushes the new `blob` securely to `.pes/objects/XX/`. Concurrently, the GC cycle wakes up natively! Because the newly minted blob has not yet actively been bound to an established `.pes/refs/heads` branch parent commit link locally, the secondary GC thread inherently views it as deeply orphaned, sweeping and deleting the fresh blob completely offline. The `commit` completes natively, tying a hard pointer locally directly to a permanently deleted element, inducing data corruption! **Git's Solution:** Real Git inherently avoids this by implementing strict timestamp bounding! The underlying garbage collector implicitly strictly ignores any structurally orphaned object newly written dynamically within a set tolerance window structurally (usually bounds less than 2 physical weeks). Natively it safely skips active pipeline data.